SECURITY AUDIT

Lista Dao

V3 Collateral

FINAL REPORT

Oct 2026

BAILSEC.IO

Disclaimer:

BailSec was engaged by the client to perform a time-boxed technical security assessment of the target identified in this report. The assessment applies only to the scope, version, commit, deployment, materials and information identified in the report. Its findings are point-in-time and not exhaustive. Changes made after the assessment—including remediation changes—were not assessed unless expressly stated otherwise and may affect the applicability of the report's conclusions.

No security assessment can identify every vulnerability or guarantee that a system is secure, error-free or protected against attack or loss. Only the components, assumptions, threat models and risk categories expressly identified as in scope were assessed. The report should be read in full, including its scope, exclusions, unresolved findings and remediation status. The client remains responsible for design, testing, remediation, deployment, operation, monitoring and incident response.

This report is a technical assessment, not a certification, attestation, investment recommendation, or legal, financial, regulatory or tax advice. It does not constitute an endorsement of the client, the assessed system or any related project.

This report was prepared for the client under a separate engagement agreement. If published, its public availability is for transparency and informational purposes only. To the fullest extent permitted by applicable law, publication does not create an auditor-client relationship, duty of care or right of reliance in favor of any third party. Other readers should conduct their own diligence and obtain appropriate professional advice.

Only the complete, unmodified report as issued by BailSec is authoritative. The signed engagement agreement exclusively governs the contractual rights and obligations between BailSec and the client, including warranties, remedies, liability limitations, indemnities and dispute resolution. Nothing in this notice excludes liability that cannot lawfully be excluded.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Lista Dao – V3 – Audit Report
Website	lista.org
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/lista-dao/moolah/blob/f7c58cb52a24f12ce6881b754d2353e904d9fbb2/src/provider/v3/
Resolution 1	https://github.com/lista-dao/moolah/tree/c9f1022d70614b8f67543a301d1fb7d30f4d0f7a

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High	3	2		1		
Medium	6	4		2		
Low	22	6		16		
Informational	2	1		1		
Governance	1			1		
Total	34	13		21		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

Base

V3Provider

The V3Provider contract is an abstract, upgradeable vault that integrates a concentrated-liquidity position with the Moolah lending protocol. It issues fungible shares representing proportional ownership of the managed liquidity, idle token balances, and accrued trading fees. These shares serve as collateral in compatible Moolah markets. The provider handles share accounting and collateral operations, while a separate V3DexAdapter holds the position NFT and executes interactions with the underlying decentralized exchange.

The deposit function accepts the pool's two underlying tokens and supports native currency deposits when one token is the configured wrapped-native asset. Share issuance follows two distinct paths. When the total share supply is zero, the provider forwards the supplied tokens to the adapter for liquidity addition. It values the resulting liquidity using the adapter's fair-price composition and resilient oracle prices, then converts this value into accounting-asset units to determine the initial share issuance.

When shares already exist, deposits instead follow the position's current token composition, including idle balances and accrued fees. The provider determines the proportional contribution supported by the supplied amounts and issues the corresponding fraction of the existing share supply. Accepted tokens are credited to the adapter's idle inventory for subsequent deployment. Both deposit paths enforce caller-specified minimum amounts and a minimum share output. Newly minted shares are supplied directly to Moolah as collateral for the designated beneficiary, and unused inputs are refunded to the caller.

The contract provides separate functions for withdrawing underlying assets and moving existing shares. The withdraw function removes shares from a user's Moolah collateral position, burns them, and instructs the adapter to return the corresponding underlying assets. The withdrawShares function releases collateral as provider shares without redeeming the underlying position. Conversely, supplyShares deposits existing shares into Moolah without issuing additional shares. Shares held outside Moolah can be redeemed through redeemShares. Collateral withdrawals require authorization from the account owner and remain subject to Moolah's collateralization checks.

Strategy maintenance is delegated to authorized BOT operators. The compound function instructs the adapter to collect fees and deploy available inventory, subject to the supplied token minimums and adapter checks. A shared internal rebalance function additionally compares fair-price net asset value before and after execution, enforcing configurable per-rebalance and cumulative daily loss limits. These provider-level loss limits apply to rebalancing; standalone compounding follows its separate execution path.

The contract exposes composition, deposit-preview, redemption-preview, and total-asset views. Total assets are expressed in the configured accounting asset using fair-price composition and resilient oracle valuations. A separate provider oracle supplies the collateral valuation used by Moolah. Although V3Provider inherits ERC-4626 accounting functionality, its standard single-asset deposit, mint, withdraw, and redeem entry points are disabled in favor of the specialized interfaces.

Administrative permissions are separated between the default administrator, MANAGER, and BOT roles. The administrator authorizes upgrades, while MANAGER controls BOT membership, deposit-whitelist settings, and rebalance loss limits. When enabled, the deposit whitelist requires both the caller and beneficiary to be approved. External ERC-20 share transfers are restricted to Moolah. An overridable collateral-change hook allows derived providers to synchronize additional accounting after collateral operations and Moolah liquidation callbacks.

Privileged Functions

- setDepositWhitelistEnabled
- setDepositWhitelist
- compound
- setMaxRebalanceLossBPS
- setMaxDailyLossBPS

Issue_01	Zero-supply reset via Moolah flashloans
Severity	High
Description	`Moolah.flashLoan()` temporarily transfers tokens held by Moolah and requires the same token quantity to be returned. This operation does not reduce the collateral quantities recorded in users' lending positions. For provider-share collateral, token quantity alone does not preserve the economic value of those recorded claims. The underlying backing per share can change independently of the number of tokens returned. `V3Provider.redeemShares()` permits wallet-held shares to be redeemed without changing Moolah's collateral records. If the entire outstanding share supply becomes redeemable, the ERC-20 supply can reach zero while Moolah continues to record collateral claims against the provider. A subsequent deposit then enters the `totalSupply() == 0` issuance branch. That branch determines share issuance from the new contribution's accounting-asset value and does not preserve the backing per share that existed before the reset. Where the previous share value had increased above this opening issuance basis, restoring the same share quantity can require less underlying backing. The accounting mismatch is therefore between Moolah's unchanged collateral quantities and the provider's restarted share valuation. During the zero-supply state, the outstanding claims are lending-account records; there are no remaining unburned ERC-20 shares. Recorded collateral claims can remain unchanged in quantity while becoming backed by fewer underlying assets. This transfers value away from the affected collateral holders and reduces their collateralization. Borrowers may consequently become liquidatable, and sufficiently large reductions can leave inadequate collateral to cover outstanding

	debt. For the restart-based extraction described above, the affected value is the backing above the reopening issuance basis, which can include accumulated trading fees. However, the mismatch between live share supply and Moolah's recorded collateral quantities creates additional exposure. While the live supply is temporarily reduced, an externally funded liquidity addition to the tracked NFT increases the backing allocated to each remaining share. `V3ProviderOracle.peek()` calculates its price using that reduced supply, while Moolah applies the resulting price to unchanged collateral records. A borrower can consequently obtain debt against a temporarily inflated collateral quotation. Restoring the share supply normalizes the quotation but does not reverse the borrowing or require every existing position to pass another health check. This can leave lenders exposed to bad debt even where the shares had not appreciated before the operation. Completion requires an eligible, separately withdrawable position for restoring the flash-borrowed shares. The overborrowed position cannot itself release collateral in violation of Moolah's health check. Share-flash-loan availability, deposit and borrowing permissions, and any applicable provider hooks must also permit the operation. For `SlisBNBV3Provider`, the same supply mismatch affects reward accounting. A small positive supply combined with additional recognized backing can inflate `getUserBalanceInBnb()` and cause synchronization to mint excessive slisBNBx. Conversely, when the supply reaches zero, that function returns zero even though Moolah still records the user's collateral, allowing permissionless synchronization to burn the associated allocation. Restoring provider supply does not automatically synchronize every affected account. Accounts not resynchronized during restoration can therefore retain excessive allocations or remain below their correct allocation until a subsequent successful synchronization. Reserve capacity must also be sufficient for the required minting and restoration hooks. These effects concern incorrect slisBNBx

	accounting; any resulting external reward loss depends on whether the reward system consumes those balances before correction. A related rounding variant can produce the same backing-reduction effect without reducing the share supply to zero. After nearly all flash-borrowed shares are redeemed, a tiny positive supply and residual NFT liquidity can remain. `V3DexAdapter.positionAmountsAt()` rounds the token amounts represented by that liquidity down to whole token base units. At this boundary, the rounding can materially understate the residual liquidity's proportional value. The next deposit consequently enters the subsequent-deposit branch. `V3ProviderLib.quoteDeposit()` uses the rounded residual composition and remaining share supply to calculate issuance. A large contribution scales against this understated composition, potentially allowing the borrowed share quantity to be recreated with substantially less backing than was removed. Rounding accepted token amounts upward and issued shares downward does not recover precision already discarded from the residual composition. The existing nonzero-USD-value check also does not exclude this condition when the rounded token amounts still have a positive USD valuation. Returning the original share quantity satisfies flash-loan repayment while Moolah's unchanged collateral records become backed by fewer underlying assets. Unlike the zero-supply variant, this mechanism does not depend on restarting initial share issuance or on prior appreciation above the opening share price. This variant shares the requirement that the relevant provider shares can be borrowed and redeemed while external claims remain outstanding. Blacklisting the provider-share token from `Moolah.flashLoan()` blocks the described route. Ordinary withdrawals alone do not establish the same large-scale impact because they remove the withdrawing users' collateral claims.
Recommendations	Consider a) blacklisting the share token and b) executing a governance deposit and withdrawing the shares from Moolah to make sure Moolah never holds the full supply of V3Provider shares.

Comments / Resolution	Resolved, the client will blacklist the share token. It has to be noted that this resolution is based on configurational expectations. If these are not indeed applied on-chain, then this issue is still exploitable.

Issue_02	Inflation attack / provider brick via NFPM donation
Severity	Medium
Description	`V3Provider` calculates subsequent deposit issuance from the adapter's recognized token composition and the existing provider-share supply. `V3DexAdapter.positionAmountsAt()` reads the liquidity associated with its tracked `tokenId` directly from the position manager. Where the position manager permits third-party `increaseLiquidity()` calls, externally funded liquidity additions to this NFT increase the provider's recognized backing without minting corresponding provider shares. This increases the backing represented by each existing share. The relevant operation is an increase to the tracked NFT. Minting a separate NFT does not update the adapter's `tokenId`, and transferring ERC-20 tokens directly to the adapter does not automatically increase its tracked idle balances. When the outstanding supply consists of very few indivisible share units, an increase in backing can make share rounding economically significant. `V3ProviderLib.quoteDeposit()` rounds the accepted token quantities upward and the issued share quantity downward. The accepted contribution is not subsequently reduced to reflect a fractional share entitlement discarded during the final share calculation. Consequently, a depositor can contribute assets for an entitlement that rounds down to a materially smaller positive number of shares. The resulting value remains in the portfolio and benefits the existing shareholders. The `ZeroShares()` check prevents deposits that receive no shares. It does not eliminate losses arising from positive but materially rounded-down issuance. The separate `minShares` check protects against this outcome when the caller supplies a sufficiently restrictive threshold; the exposure therefore requires an absent or insufficient share

	minimum. This boundary is not limited to the original deployment. Partial withdrawals can leave a very small positive supply. An intervening rebalance replaces the managed NFT without establishing a minimum remaining share supply, so it does not inherently remove the condition. Any external liquidity addition must affect the currently tracked NFT. A complete exit to zero supply instead selects a different deposit branch and should not be treated as the same inflation mechanism. The earlier resolution combined `minShares` with operational dead-share seeding. Accordingly, the remaining concern is conditional on an insufficient persistent seed and inadequate caller protection. Due to the minShares parameter, the main impact is essentially bricking the provider due to creating an unreasonable exchange rate.
Recommendations	Consider executing the first deposit as governance.
Comments / Resolution	Resolved, the client will execute the first deposit. It has to be noted that this resolution is based on configurational expectations. If these are not indeed applied on-chain, then this issue is still exploitable.

Issue_03	Temporary unaccounted inventory during rebalance exposes collateral to incorrect liquidation pricing
Severity	Medium
Description	During a full rebalance, `V3DexAdapter.rebalance()` collects the position's assets, clears `idleToken0` and `idleToken1`, burns the previous NFT, and resets `tokenId` before performing the optional inventory conversion. At this point, the recovered inventory is represented by the local `total0` and `total1` variables. However, `positionAmountsAt()` does not include these variables in its valuation. With no active NFT and zero recorded idle balances, it returns zero for both tokens despite the continued existence of the portfolio's assets. This temporary accounting state is exposed during the external call to the whitelisted conversion venue. With positive provider-share supply and valid underlying feeds, `V3ProviderOracle.peek()` converts the zero valuation into its minimum price of `1`, corresponding to $0.00000001 per whole share. If the approved venue's execution permits a callback into Moolah during this interval, a market using the affected oracle can observe this temporary price when evaluating collateral health and liquidation amounts. The provider and adapter's reentrancy guards do not prevent valuation reads or independently lock Moolah. The provider's liquidation notification is also callable while its rebalance guard is active. The resulting concern is liquidation based on temporarily understated collateral value rather than an actual deterioration in backing. A liquidation transfers existing provider shares and updates Moolah's accounting; it does not inherently reduce the adapter's assets or the provider's total share supply. Consequently, the final rebalance NAV checks need not detect this effect. Once the replacement position and remaining idle balances are recorded, the reported portfolio value can recover while the collateral

	ownership changes caused by the liquidation remain effective. Successful exploitation is conditional on an actual callback-capable route through an approved venue, the market's liquidation permissions, and completion of the applicable repayment, provider-hook, and final transaction checks. Immediate redemption during the rebalance is separately restricted by the active reentrancy guards. A compromised BOT or erroneous swap calldata alone does not establish a usable callback route. The temporary valuation inconsistency is confirmed by the accounting flow. A claim of successful near-zero-priced liquidation requires validation against the deployed venue and liquidation integration; it cannot be established from whitelist membership alone.
Recommendations	Consider ensuring that no callback can ever be executed during this state. Moreover, consider not falling back to a price of 1 when the price is zero. Reverting instead is more appropriate, even if that means liquidations may not succeed in case of dust valuation. If that theoretical case ever happens, an upgrade can be made to restore the price = 1 path and hot-fix liquidations. However, we consider more theoretical risk from a fallback price of 1 compared to a revert of liquidations, as the latter can result in a loss of funds.
Comments / Resolution	Resolved, a reentrancy guard has been applied.

Issue_04	Withdrawal previews can overestimate execution proceeds because of different rounding sequences
Severity	Low
Description	`V3DexAdapter.previewRemoveLiquidity()` calculates the expected withdrawal amounts by applying the holder's proportional share to the position's combined token composition. This composition includes deployed principal, accrued fees, and tracked idle balances. The preview rounds down the resulting aggregate entitlement once for each token. The actual withdrawal applies a different rounding sequence. `removeLiquidity()` first collects fees into idle inventory, rounds down the proportional liquidity to remove, obtains the rounded token amounts resulting from that liquidity removal, and separately rounds down the holder's entitlement to each idle balance. These independently rounded components are then combined to determine the amounts delivered. **Consequently, actual withdrawal proceeds can be smaller than the preview even when the pool price, position state, and share supply have not changed.** An integration that uses the previewed amounts directly as `minAmount0` and `minAmount1` can therefore encounter `InsufficientAmount()` without any adverse price movement or intervening transaction. This is particularly relevant because `previewRedeemUnderlying()` is documented as supporting the selection of tight minimum amounts. The affected operation reverts atomically, including the provider-share burn and liquidity removal. The discrepancy does not transfer value to an attacker or cause a successful withdrawal to receive less than its enforced minimums. The impact is failed tightly quoted withdrawals or redemptions and additional integration handling for a rounding difference that the preview does not reproduce. The liquidator's redemption functions accept BOT-supplied minimums rather than automatically consuming the preview. Their exposure therefore depends on whether the keeper uses these preview values

	without accounting for the execution difference.
Recommendations	Consider adding NATSPEC to highlight this behavior.
Comments / Resolution	Acknowledged.

Issue_05	Markets without provider registration can bypass collateral synchronization and share-recipient restrictions
Severity	Low
Description	`V3Provider.deposit()` verifies that the supplied market uses the provider's share token as collateral, but does not verify that Moolah has registered the provider for that market. Moolah accepts collateral supplied into an existing market whose provider entry is unset. Consequently, the V3 provider can deposit shares into such a market and execute its normal collateral-change hook even though subsequent withdrawals are not required to pass through the provider. For `SlisBNBV3Provider`, the deposit updates `userMarketDeposit` and `userTotalDeposit` and synchronizes the corresponding slisBNBx allocation. However, a direct withdrawal through `Moolah.withdrawCollateral()` decreases the actual collateral position without executing that synchronization hook. Actual market collateral can therefore become zero while the provider continues recording a positive deposit. The returned wallet shares can also be redeemed through `redeemShares()`, which does not clear the stale market record. While provider share supply remains positive, `getUserBalanceInBnb()` continues valuing the cached deposit against the remaining portfolio. The associated slisBNBx allocation can therefore persist after the collateral supporting that account has been withdrawn. This mismatch can also consume reserve-mint capacity. Where the module has a nonzero fee rate, stale allocations retain balances in the minter's MPC accounting. Exhausted capacity can cause operations requiring additional reserve issuance to revert with `EXCEED_MPC_CAP`, including operations from other modules sharing the same minter. This does not imply that every withdrawal or liquidation becomes blocked.

	The same registration gap weakens the share-recipient restriction. Direct Moolah withdrawals permit a chosen receiver, and the share token accepts the transfer because its immediate caller is Moolah. This bypasses the provider's `withdrawShares()` restriction requiring self-receipt when its deposit whitelist is enabled. Only legitimately owned or authorized collateral can be moved. The market must exist; arbitrary nonexistent `MarketParams` are rejected. Market creation is permissionless only when Moolah has no OPERATOR members. Otherwise, creation requires that role. Removing an existing provider registration can also create the relevant state. The accounting discrepancy is recoverable through `SlisBNBV3Provider.syncUserBalance()` for the affected account and market. Synchronizing only the minter does not repair it because the minter reads the provider's stale cached balance.
Recommendations	Consider ensuring that only compatible markets are created and market creation remains permissioned.
Comments / Resolution	Resolved. If an existing Moolah market uses the V3 provider's shares as collateral **with its provider entry unset**, users can supply shares directly and have the position credited through permissionless `SlisBNBV3Provider.syncUserBalance()`, which validates only the collateral token. With an enabled and authorized minter, a subsequent direct `Moolah.withdrawCollateral()` bypasses provider synchronization, leaving cached collateral and the corresponding slisBNBx allocation outstanding after the collateral exits. The registration checks added to `deposit()` and `supplyShares()` therefore leave this route open; **correctly registered markets reject the direct entry**.

Issue_06	Idle entry and proportional redemption can reduce deployed liquidity
Severity	**Low**
Description	For deposits made while shares already exist, `V3Provider.deposit()` transfers the accepted tokens to the adapter and records them through `creditIdle()`. The deposit does not increase the liquidity held by the existing position NFT. Nevertheless, the newly issued shares represent proportional ownership of the complete managed position, including both the newly credited idle inventory and liquidity deployed before the deposit. The withdrawal mechanism settles this combined claim proportionally. `V3DexAdapter.removeLiquidity()` calculates the withdrawing holder's fraction of existing NFT liquidity, removes that liquidity, and distributes the same fraction of idle balances. It does not distinguish between shares issued against idle inventory and shares backed by previously deployed liquidity. Consequently, redeeming shares from an idle-only deposit can remove liquidity that the deposit never added. The remaining holders retain a larger portion of their backing as idle tokens and a smaller portion as deployed liquidity. This can be repetitively executed to grief liquidity out of the active system and serves as required precondition for a more severe issue (BOT can drain funds).
Recommendations	Consider acknowledging this condition.
Comments / Resolution	Acknowledged.

Issue_07	Deposit minimums do not preserve expected refunds when desired amounts include surplus
Severity	Low
Description	When desired amounts include surplus tokens, the minimum-consumption checks do not independently preserve the quoted entry composition or expected refund. A change in the portfolio's live composition can increase consumption of the surplus token while leaving consumption of the limiting token unchanged. Both token minimums can therefore remain satisfied, even when they equal the originally quoted consumption. The `minShares` check does not necessarily reject this change. Additional token consumption can represent a larger proportional deposit and produce more shares. The original absolute share floor remains satisfied without constraining the relationship between additional assets committed and additional shares received. Consequently, an integration that presents these minimums as a strict economic slippage tolerance may imply stronger protection than the contract provides. The contract enforces the supplied consumption bounds, but does not guarantee the quoted refund or token ratio. Consider the following illustrative portfolio snapshots. Share supply remains unchanged before the deposit, and rounding is omitted: **Existing portfolio** / **At quotation** / **At execution** Token0: 600 / 1,000 Token1: 600 / 300 Provider shares: 1,000 / 1,000 The user offers **600 token0 and 150 token1**.

At quotation, token1 limits the deposit. The portfolio holds equal quantities of both tokens, so the quote accepts **150 token0 and 150 token1**, issues **250 shares**, and refunds **450 token0**.

The user selects minimums matching that quote exactly:

Parameter	Value
`amount0Min`	150 token0
`amount1Min`	150 token1
`minShares`	250 shares

At execution, the portfolio instead contains 1,000 token0 and 300 token1. The offered 150 token1 covers half of that portfolio composition. The corresponding deposit therefore accepts **500 token0 and 150 token1** and issues **500 shares**.

Result	Original quote	Execution
Token0 consumed	150	500
Token1 consumed	150	150
Shares issued	250	500
Token0 refunded	450	100
Token1 refunded	0	0

	All three minimums pass. Nevertheless, the user commits **350 more token0** and receives **350 fewer token0 as a refund** than originally quoted. The additional contribution also produces **250 additional shares**. These figures demonstrate the change in commitment and refund; they do not establish a financial loss.
Recommendations	Consider implementing NATSPEC which highlights this behavior.
Comments / Resolution	Acknowledged.

Issue_08	WBNB refunds are unconditionally converted into native BNB
Severity	Low
Description	`V3Provider.deposit()` supports funding the wrapped-native leg through either WBNB transfers or native BNB. However, the refund mechanism does not preserve the asset form supplied by the caller. Any unused WBNB is returned as native BNB, including deposits made entirely through ERC20 transfers with `msg.value == 0`. For initial deposits, the adapter unwraps unused WBNB and returns native BNB to the provider. For subsequent deposits, the provider unwraps the unused WBNB itself. Both branches ultimately call `_refund()`, which sends native BNB to `msg.sender`. Consequently, a contract that supports ERC20 transfers but rejects native BNB cannot complete a deposit whenever the wrapped-native leg produces a nonzero refund. The same restriction affects callers that accept native currency only from specific senders, such as the WBNB contract itself. For example, a contract supplies **10 WBNB**, but the deposit consumes only **9.5 WBNB**. The remaining **0.5 WBNB** is unwrapped and sent back as **0.5 BNB**. If the caller rejects this payment, `_refund()` reverts with `BnbTransferFailed()`. The entire deposit then rolls back, including token transfers, share issuance, collateral supply, and any liquidity changes. The direct impact is therefore integration incompatibility and failed deposits, rather than loss of the refunded assets. A zero-refund deposit avoids this callback, but changes in execution composition or rounding can turn an expected zero refund into a nonzero amount. Supplying an `onBehalf` address that accepts native BNB does not resolve the incompatibility because refunds are sent to the caller. The adapter also uses unconditional native settlement for the wrapped-native leg of withdrawals, making this behavior relevant to integrations

	supporting the complete deposit-and-withdraw lifecycle.
Recommendations	Consider adding NATSPEC for this behavior.
Comments / Resolution	Acknowledged.

Issue_09	A fixed daily loss allowance can prevent economically acceptable large rebalances
Severity	Low
Description	`V3Provider._guardedRebalance()` limits cumulative daily losses using `maxDailyLossUsd`, which defaults to $1,000. This allowance is expressed as an absolute USD amount and does not automatically increase with provider TVL or the amount being converted. Conversely, ordinary conversion costs generally increase with transaction size. Selling an LST below its accounting reference, paying trading fees, or incurring execution slippage can produce a small percentage loss that exceeds the remaining absolute daily allowance. For example, assume a rebalance converts assets valued at $600,000 and incurs a 0.2% discount relative to the accounting reference. The conversion represents a $1,200 value difference. If this becomes the complete rebalance's net fair-NAV decrease, without offsetting valuation effects, the transaction exceeds the default daily allowance and reverts with `DailyLossExceeded`, even if the separate percentage-based checks pass. The daily check measures the net fair-NAV change across the full rebalance. A gross conversion discount does not necessarily cause rejection if other valuation changes offset it. However, where the net decrease exceeds $1,000, an otherwise acceptable conversion cannot complete under the default configuration. Splitting the operation into multiple transactions during the same UTC day does not avoid the cumulative limit. Moreover, a transaction whose own net loss exceeds the entire daily allowance remains too large even immediately after a daily reset. The impact is an operational restriction on rebalancing as the provider grows. Rejected transactions roll back completely, preserving the previous position and assets. This is a configuration consideration rather than an accounting bypass or a realized loss of principal.

Recommendations	Consider scaling the daily loss limits based on the overall TVL.
Comments / Resolution	Acknowledged.

Issue_10	First deposit should be made by governance
Severity	**Informational**
Description	`V3Provider.deposit()` uses a separate issuance mechanism when `totalSupply()` is zero. This path adds liquidity through the adapter and determines the initial share supply from the fair-price valuation of the newly added liquidity, expressed in the accounting asset. The contract does not reserve the initial deposit for governance or enforce an economically meaningful minimum share supply. Once deposits are accessible, an eligible caller can establish the opening position with any contribution that passes the liquidity and share-output checks. Although `ZeroShares()` rejects contributions that round down to zero shares, it does not prevent a very small positive supply. Similarly, `minShares` protects the individual depositor's requested output but does not establish a protocol-wide minimum supply. Consequently, the previously identified first-depositor and small-supply risks remain dependent on the deployment procedure. It does also eliminate further unexpected edge-cases if governance executes the first deposit and permanently keeps that stake.
Recommendations	Consider executing the first deposit.
Comments / Resolution	Resolved, governance will execute the first deposit.

V3DexAdapter

The V3DexAdapter contract is an abstract, upgradeable component responsible for custody and management of a concentrated-liquidity position. It holds the position NFT, maintains the active tick range, tracks idle token balances, and executes interactions with the position manager. The associated V3Provider manages share issuance and Moolah collateral accounting, while the adapter manages the underlying assets.

The adapter is configured for a specific token pair, fee tier, position manager, and liquidity pool. Its constructor validates token ordering and verifies that the corresponding pool exists. The authorized provider is assigned through a one-time administrative function, which verifies that the provider references the same adapter. Liquidity additions, removals, idle-balance credits, compounding, and rebalancing are restricted to this provider and protected against reentrancy.

The addLiquidity function creates a position when no NFT exists or increases liquidity in the existing position. Unused inputs are returned to the designated refund recipient. Separately, creditIdle records tokens transferred by the provider without deploying them into the pool and verifies that the recorded inventory is supported by the adapter's token balances. Wrapped-native tokens are unwrapped when distributed through refund and withdrawal operations.

The removeLiquidity function settles withdrawals proportionally across deployed liquidity and idle inventory. It first collects accrued fees into idle balances, then removes the requested fraction of liquidity and distributes the corresponding fraction of idle assets. Minimum-output checks apply to the combined amounts delivered to the recipient. A withdrawal that would deliver neither token despite redeeming a positive number of shares reverts.

Compounding collects trading fees and combines them with existing idle balances for reinvestment into the current position. Before increasing liquidity, the adapter checks the pool's spot price against its fair-price reference, unless this check is disabled. If the deviation exceeds the configured tolerance, or the available inventory cannot produce additional liquidity, the assets remain idle. Successful compounding retains any unused tokens as idle inventory.

Rebalancing derives a new tick range, collects fees, withdraws the existing liquidity, and replaces the position NFT. The combined assets may undergo an inventory conversion through a whitelisted external venue before the replacement position is minted. Conversion accounting uses measured token-balance changes and enforces the supplied output minimum. Where a nonzero reference rate is available, an additional rate-based swap-loss limit applies. Rebalancing also supports a deadline, minimum minted liquidity, an optional target-price check, and configurable checks on reference-rate changes. A restricted same-range path compounds into the existing position instead of replacing it.

The adapter exposes views for position composition, liquidity, prices, and operation previews. Position composition includes principal, tracked idle balances, recorded amounts owed, and pending fees calculated from pool fee growth. The base fair-price implementation uses a

subclass-provided exchange rate when available and otherwise falls back to the pool's time-weighted average price. MANAGER controls range parameters, deviation thresholds, permitted swap venues, and swap-loss limits, while the default administrator authorizes upgrades.

Privileged Functions

- setProvider
- addLiquidity
- removeLiquidity
- collectAndCompound
- creditIdle
- setCenterRateThresholdBps
- setMaxSpotDeviationBps
- setRangeBps
- setMaxCenterRateDeviationBps
- setSwapPairWhitelist
- setMaxSwapLossBps
- rebalance

Issue_11	Compound and deposit<->withdraw sequence allows malicious BOT to repetitively drain funds
Severity	High
Description	`V3Provider.compound()` allows the BOT to deploy accumulated fees and idle inventory into the existing liquidity position. Unlike rebalancing, this operation does not pass through `_guardedRebalance()` and therefore does not enforce the provider's per-operation NAV limit or cumulative daily loss allowance. The adapter instead checks whether the execution price falls within `maxSpotDeviationBps` of its fair-price reference and forwards the BOT-supplied token minima to the position manager. The default deviation allowance is **50 basis points**. Consequently, execution at an unfavorable price can remain valid provided it falls inside this interval and satisfies the supplied minima. A compromised BOT controls both the timing of compounding and its token-consumption minima. These caller-controlled parameters therefore do not independently protect the provider against that role. Any resulting loss from adverse deployment is also not charged against `dailyLossAccum`. This appears usually fine, as it does not seem to be standard behavior to compound multiple times in a short period and as such, the impact from such a malicious action is limited. The previously identified idle-entry/proportional-exit behavior can make additional assets available for compounding. An eligible depositor can complete a value-conserving deposit and redemption while leaving part of the remaining holders' assets idle. This connects the allocation issue to the compounding exposure, although possession of the BOT role does not itself bypass deposit eligibility requirements. Where adverse deployment produces economically profitable arbitrage, existing holders bear the resulting reduction in portfolio value. Repeated exposure is possible when deployable inventory remains available or is replenished. However, simply calling `compound()` repeatedly does not reproduce the same effect after the available inventory has been consumed. Here comes the idea of depositing and

	withdrawing into play, to artificially produce idle assets. Restricting the interval between compounds would limit execution frequency, but would not remove the underlying acceptance of unfavorable execution prices.
Recommendations	Consider restricting the compound as well as no-op rebalance.
Comments / Resolution	Resolved.

Issue_12	Realizing the SPOT vs FAIR composition gap can conceal swap losses from the daily-loss accumulator
Severity	Medium
Description	`V3Provider._guardedRebalance()` compares the portfolio's fair NAV before and after rebalancing. The daily-loss accumulator is increased only when the ending value is lower than the starting value. Consequently, value increases recognized within the same operation offset losses before the amount charged to `dailyLossAccum` is determined. The valuation treats invested liquidity and idle inventory differently. Invested liquidity is decomposed into token amounts at the adapter's fair price, whereas idle inventory is included using its actual token quantities. When spot differs from fair, the actual token basket recoverable by removing liquidity can have a different value from the fair-composition snapshot. **The valuation direction is important: this mechanism does not require reported NAV to exceed actual backing. The starting fair-composition valuation can be lower than the value of the actual redeemable basket.** Removing liquidity and retaining more of the recovered assets as idle inventory can recognize that previously unrepresented difference. If an inventory conversion incurs a loss during the same rebalance, that recognition can offset the conversion cost. The ending fair NAV may therefore equal or exceed the starting fair NAV even though the actual recoverable token value decreased. This requires the **final portfolio**, after replacement minting, to recognize enough of the composition difference. Merely removing the NFT temporarily is insufficient if the subsequent remint restores an equivalent valuation gap. This gap can be abused by a compromised BOT role to rebalance with unfavorable swaps while concealing the losses under the described scenario.

Recommendations	This issue is inherent to the way how the NAV check works. We recommend ensuring that the BOT remains fully/partially trusted.
Comments / Resolution	Acknowledged.

Issue_13	Range changes or price movements before the first deposit do not update the opening position
Severity	Low
Description	During initialization, `V3DexAdapter` derives the opening position's tick boundaries from the initial configuration and stores them in `tickLower` and `tickUpper`. A subsequent call to `setRangeBps()` updates `rangeLowerBps` and `rangeUpperBps`, but does not recalculate those stored tick boundaries. This remains true when no position has been created and `tokenId` is still zero. When the first deposit reaches `addLiquidity()`, the adapter mints the position using the previously stored ticks. It does not derive the opening range again from the updated configuration. For example, an adapter initialized with nominal **1% margins** can later be configured with **0.2% margins** before accepting any deposits. Unless a rebalance updates the stored ticks first, the opening position still uses the original boundaries, subject to tick alignment. The configured margins and the actual opening position therefore disagree. Assets can be deployed with a different concentration, active-price interval, and token-consumption profile from those intended by the manager. Deposit minima may reject some resulting differences, but they do not directly enforce the requested tick range. Applying the new configuration requires a successful rebalance. For the slisBNB and stable variants, this can be delayed by the center-rate movement requirement: changing the range settings does not reset `lastCenterRate` or bypass `_requireCenterRateDeviation()`. Consequently, even an empty adapter may be unable to apply its updated opening configuration through an ordinary BOT rebalance while the center rate remains unchanged. This is a configuration-application inconsistency. It does not, by itself, establish asset loss, and the stored tick getters still expose the

	boundaries that the first mint will actually use. The same initialization-to-first-deposit gap also exists without any configuration change. `tickLower` and `tickUpper` are calculated during initialization, but the first call to `addLiquidity()` uses those stored boundaries without recalculating them against the current center rate. Ordinary movement in the stablecoin oracle ratio or the slisBNB conversion rate can therefore make the opening range outdated before any liquidity has been deposited. For example, assume initialization occurs at a center rate of 1.00 with configured margins of approximately 0.2%, producing boundaries near 0.998–1.002. If the reference rate and pool price subsequently reach 1.01 before the first deposit, the adapter still opens the position using the original boundaries unless a successful rebalance has refreshed them. The opening position can consequently be immediately out of range and consume only one token, refunding the other. This outcome requires the depositor's amount and share minimums to permit it; otherwise, the deposit reverts.
Recommendations	Consider keeping this footgun in mind.
Comments / Resolution	Acknowledged.

Issue_14	Intervening deposits weaken the keeper's absolute compounding minimums
Severity	Low
Description	The keeper prepares `amount0Min` and `amount1Min` against the inventory expected to be available for compounding. However, the operation deploys the inventory available **at execution**, which can include deposits accepted after the keeper prepared or broadcast the transaction. Subsequent deposits enter the adapter as tracked idle inventory. Their amounts and share credit are derived from the existing live composition, but they do not update the parameters of an already-pending compound. The result is a mismatch between the inventory used to calculate the minimums and the inventory against which those minimums are enforced. **A minimum that tightly constrains a smaller operation can become permissive when additional inventory increases the absolute amounts consumed.** The same execution price may fail the keeper's limits without the intervening deposit and pass them afterward. **The same is applicable for rebalancing and amountMin values.**
Recommendations	Consider keeping that in mind and ensuring that all other slippage checks remain strict.
Comments / Resolution	Acknowledged.

Issue_15	Rebalances losses can trigger borrower liquidations
Severity	Low
Description	`V3DexAdapter.rebalance()` replaces the managed liquidity position and may convert part of its underlying inventory. The provider's `_guardedRebalance()` permits this operation to decrease the portfolio's fair NAV within configured limits. By default, the measured loss must remain within both 2% of the pre-rebalance NAV and the remaining $1,000 cumulative daily allowance. These protections constrain aggregate portfolio losses but do not preserve the health of individual borrowing positions. Provider shares are used as collateral in Moolah. `V3ProviderOracle.peek()` values these shares using the adapter's current underlying assets and the outstanding share supply. A rebalance that reduces backing without changing supply can therefore reduce the collateral value reported to Moolah, even when external token prices remain unchanged. For example, a provider holding $100,000 may incur an accepted $1,000 rebalance loss. A borrower whose collateral was previously valued at $10,000 would consequently hold collateral valued at $9,900. At an 80% liquidation threshold, the supported debt decreases from $8,000 to $7,920. A position carrying $7,950 of debt was healthy before the rebalance but becomes eligible for liquidation afterward. The borrower can consequently incur both the proportional portfolio loss and an additional liquidation penalty. The rebalance loss limits do not cap the amount of collateral subsequently liquidated or the resulting liquidation incentive. A later recovery in share value does not reverse a completed liquidation. This consequence requires an accepted rebalance loss, insufficient borrower headroom, and a subsequent valid liquidation. It represents an economic risk of using actively managed liquidity shares as collateral, rather than an incorrect oracle calculation or a bypass of the rebalance loss limits.

Recommendations	Consider accepting this risk.
Comments / Resolution	Acknowledged.

Issue_16	Deviation checks lose relative precision at extremely small raw prices
Severity	Low
Description	`V3DexAdapter._priceWithinDeviation()` checks whether the pool price remains within a configured deviation from a reference price. Compounding uses this helper to compare spot against fair value, while full rebalances use it to compare the final spot against the BOT-provided target. Before performing the comparison, the function converts both square-root prices into integer Q96 prices. This conversion truncates precision. At sufficiently small raw token ratios, materially different prices can therefore produce the same integer representation. Calculating the permitted bounds afterward cannot recover the discarded precision. A positive raw price below approximately 1.262×10^{-29} becomes zero during this conversion. If both prices round to zero, the comparison accepts them without retaining any information about their relative difference. Prices immediately above this threshold also have coarse relative precision. The relevant quantity is the ratio between the tokens' **smallest units**, rather than their whole-token market prices. Consequently, an unusually large difference in token decimals can make this limitation relevant even when whole-token prices are comparable. In an affected configuration, the helper may accept a price outside the intended deviation threshold. This weakens the compounding price gate and the rebalance target-price assertion. Other controls remain applicable, including token minima, liquidity requirements, access restrictions, and the provider's rebalance NAV limits. The arithmetic limitation alone does not establish an economic loss. For the intended slisBNB/WBNB pair, both tokens use 18 decimals and the price ratio is around unity. Its Q96 representation is approximately 10^{29}, making integer truncation negligible. Ordinary stablecoin decimal combinations likewise retain substantial precision. Accordingly, this is a confirmed numerical limitation, but a material vulnerability requires

	establishing an affected supported configuration.
Recommendations	Consider executing tests against this issue with different configuration and acknowledge this behavior.
Comments / Resolution	Acknowledged.

Issue_17	Stale conversion instructions can complete a rebalance without achieving the intended inventory allocation
Severity	Low
Description	`_convertToOptimalRatio()` executes a backend-prepared conversion plan containing the direction, input amount, output minimum, venue, and venue-specific calldata. The function receives target tick boundaries but does not use them to calculate or validate the resulting inventory ratio. It therefore executes the supplied plan rather than independently establishing that the plan remains suitable for the current inventory and replacement range. Between preparation and execution, deposits, withdrawals, fee accrual, rate updates, or pool-price changes can alter the quantities available or the composition required by the replacement position. **An execution can be correctly priced as a swap while still being the wrong conversion for the current portfolio.** The input may be too small, too large, or in an unsuitable direction for the revised allocation. Off-chain routing is not inherently the defect. The relevant limitation is the absence of a binding relationship between the prepared conversion and the inventory or target allocation ultimately processed.
Recommendations	Consider acknowledging this behavior. Even with a strict deadline, it can always happen that the environment can change.
Comments / Resolution	Acknowledged.

Issue_18	Upper-bound rounding can eliminate the configured margin and create inactive liquidity
Severity	Low
Description	The upper endpoint is calculated by first converting the target price into an integer tick and then rounding that tick upward to the pool's tick spacing. **The first conversion can discard information that the second rounding step cannot recover.** If the price-to-tick conversion returns a tick already aligned to the spacing, `_ceilTick()` leaves it unchanged—even when the intended upper price lies above that tick. There is an additional precision boundary in the supplied conversion: `_sqrtPriceX96FromRate()` takes a downward-rounded square root, whereas the pinned `TickMath.getSqrtRatioAtTick()` rounds its result upward. At an exact tick-price target, the former can therefore fall just below the representation used to recognize that tick. GitHub For a sufficiently narrow configured margin, the resulting upper endpoint can coincide with the center. The range remains nonempty and passes the endpoint-order check, but the requested upper-side margin has disappeared. Assume: **Parameter** — **Value** Token decimals — **18 / 18** Center rate — **1.00000** Lower margin — **1 basis point — 0.01%** Upper margin — **1 basis point — 0.01%**

<table>
<tr><td></td><td>
Tick spacing 10

The intended target prices are 0.99990 and 1.00010.

An integer reconstruction of the supplied helpers produces:

<table>
<tr><th>Calculation</th><th>Lower endpoint</th><th>Upper endpoint</th></tr>
<tr><td>Price-to-tick result</td><td>−2</td><td>0</td></tr>
<tr><td>After tick-spacing alignment</td><td>−10</td><td>0</td></tr>
</table>

For the upper target, the adapter's square-root representation is one integer unit below the value returned for tick 1 by the pinned <code>TickMath</code>. It is consequently classified as tick 0. Rounding tick 0 to spacing 10 still returns tick 0.

The final range is <code>[-10, 0]</code>, rather than a range extending above the center such as <code>[-10, 10]</code>.

Both configured margins are positive.
</td></tr>
<tr><td>Recommendations</td><td>Consider keeping this rounding case in mind when determining ranges.</td></tr>
<tr><td>Comments / Resolution</td><td>Resolved. The corrected upper-tick rounding can widen the calculated range by one tick-spacing interval, changing token requirements, obtainable liquidity, and leftover balances. A previously single-sided position may require both tokens when recreated, potentially causing rebalance to revert if the necessary inventory is unavailable. For positions created under the previous calculation, the corrected ticks may also cause <code>noOp=true</code> to revert with <code>RangeMoved</code>, or <code>noOp=false</code> to select a full rebalance instead of the shortcut, subject to the existing rate checks. Existing NFTs and stored tick boundaries are not automatically updated by this change.</td></tr>
</table>

Issue_19	Collapsed-range recovery selects the maximum-price interval regardless of the intended center
Severity	Low
Description	After aligning and clamping the calculated endpoints, `_initialTickRange()` checks whether the lower endpoint is greater than or equal to the upper endpoint. When that condition is true, it replaces the range with the final usable interval immediately below the maximum tick. **The recovery does not consider where the original range was intended to be located or which boundary caused the collapse.** This behavior is specifically identified as a flawed fix in the existing audit report: a range collapsing because the lower endpoint was raised to `minUsable` is nevertheless redirected toward `maxUsable`. The replacement is ordered and lies within valid tick bounds, but these properties do not make it a correct replacement for the intended range. This state can occur if both margins are set to zero, which is prohibited during initialization but not by the ordinary setter.
Recommendations	Consider ensuring that the condition of lower >= upper can never be true. This is a configurational topic.
Comments / Resolution	Resolved.

Issue_20	A transaction intended for the shortcut can execute a full rebalance without equivalent execution safeguards
Severity	Low
Description	The rebalance shortcut is selected only when the recalculated ticks match the existing position and the caller supplies empty swap data, zero token-removal minimums, zero minimum liquidity, and no target-price assertion. **The caller does not commit the transaction to that branch.** A change to the rate-derived ticks between preparation and execution can cause the same parameters to select the full rebalance path instead. The intended shortcut is also not a strict no-op: it may compound fees and idle assets. The security distinction is that this compounding uses `_spotWithinFair()`, whereas the full replacement mint has no equivalent independent fair-price gate. On the full path, a zero target skips the caller's target-price assertion. The result is a transaction prepared for a spot-guarded compound becoming a full-position removal and replacement with no effective caller-supplied removal or minimum-liquidity floors. Consider equal-decimal stable tokens, tick spacing **10**, and range margins of **0.5% on both sides**: **Moment** / **Center rate** / **Calculated lower tick** / **Calculated upper tick** Previous successful centering / **0.999980** / −60 / 50 Keeper prepares the transaction / **1.000000** / −60 / 50 Transaction executes after a rate update / **1.000002** / **−50** / 50 At preparation, the calculated range matches the existing NFT. The

	keeper supplies the zero-valued execution parameters required for the shortcut. At execution, the lower tick changes. The full rebalance branch is selected. The additional rate movement is only **0.002%**, so an assumed expected-rate tolerance of **0.01%** accepts it. Nevertheless, the resulting tick boundaries are different. Furthermore, NAV checks remain present.
Recommendations	Consider adding an additional parameter: bool noOp which reverts if the full rebalance is triggered while noOp is intended.
Comments / Resolution	Resolved.

Issue_21	Zero-center-rate fallback derives the replacement range from manipulable spot
Severity	**Informational**
Description	When the adapter has no nonzero center rate, `_initialTickRange()` constructs the replacement range around the pool's current tick. This means the range's location depends on the execution-time spot price rather than an independent valuation reference. **The architectural concern is that the same externally influenced pool state determines both the immediate execution composition and the location of the replacement liquidity.** A temporarily displaced spot can consequently produce a replacement range that does not correspond to the intended long-term price. Returning the pool toward its independent reference can change the position's composition, reduce its value, or leave it outside the active trading range. The normal nonzero-rate paths of the scoped specializations are different: the stable adapter derives its center from underlying oracle prices, while the slisBNB adapter obtains its center from StakeManager. **This fallback observation is not attributed to those ordinary operating paths and is merely a note for developers.**
Recommendations	No action is needed. Future modules should follow the same override approach of the _lstNativeRate function.
Comments / Resolution	Acknowledged.

Issue_22	A single-sided compound quote does not preserve expected unused-token inventory
Severity	Low
Description	`V3DexAdapter._collectAndCompound()` offers all tracked idle inventory and newly collected fees to the position manager. The BOT supplies minimum consumed amounts, but these parameters do not limit how much of an otherwise unused token may become deployed. When the pool price is at or above the position's upper boundary, a liquidity increase requires only token1. An honest BOT can therefore legitimately submit `amount0Min = 0` and a positive `amount1Min`, expecting the available token0 inventory to remain idle. If the execution price moves downward into the position's range, the liquidity increase requires both tokens. Provided sufficient token0 inventory exists, token1 can remain the limiting asset. The operation can then consume the expected token1 amount together with additional token0 and mint more liquidity. Both original minima can remain satisfied. The position manager does not hold the previously quoted liquidity amount constant; it recalculates the liquidity increase using the execution price and available inventory. The resulting difference is that token0 expected to remain idle becomes exposed to the liquidity position. The scenario requires the relevant range boundary to intersect the permitted spot-to-fair interval, sufficient inventory of both tokens, and execution that still satisfies the positive token1 minimum. If only token1 is actually available, moving into the range produces no addable liquidity, and the adapter parks the inventory instead.
Recommendations	This issue describes an edge-case where one of both amount limitations is disabled. Due to the fact that the price deviation check is still existing, this does not expose an extraordinary danger.
Comments / Resolution	Acknowledged.

Issue_23	Redemption previews can overstate deliverable amounts due to inconsistent rounding
Severity	Low
Description	`V3DexAdapter.previewRemoveLiquidity()` estimates redemption proceeds by obtaining the position's aggregate token balances, including principal, fees, and idle inventory, and then applying the requested share fraction. The actual redemption follows a different calculation order. `removeLiquidity()` first rounds down the proportional liquidity to remove, obtains the corresponding principal, and separately rounds down the withdrawing user's allocation of idle inventory and collected fees. Applying the share fraction once to aggregated balances does not always produce the same result as independently rounding each component. Consequently, the preview can overstate the amount delivered even when the pool price and relevant balances remain unchanged. For example, consider a token leg containing three base units of principal and three base units of idle inventory. A 50% redemption previews three base units. Execution can instead return one base unit from the principal and one from idle inventory, delivering only two. `V3Provider.previewRedeemUnderlying()` inherits this discrepancy. A caller that uses its output as an exact minimum can therefore encounter `InsufficientAmount` despite executing against unchanged market conditions. The same issue affects a `V3Liquidator.flashLiquidate()` transaction when its redemption minima are derived directly from the preview. Even a one-base-unit overstatement is sufficient to revert the entire operation. This causes avoidable withdrawal or liquidation failures and requires the caller to revise its execution parameters. All associated state changes revert atomically; the failed transaction does not

	permanently burn shares or transfer collateral.
Recommendations	Consider acknowledging this rounding behavior.
Comments / Resolution	Acknowledged.

V3ProviderOracle

The V3ProviderOracle contract is an upgradeable implementation of Moolah's oracle interface that values a designated V3Provider share token. It calculates an eight-decimal USD price per whole share using the underlying position's composition, resilient oracle prices, and a configurable valuation haircut. The adapter, provider share, underlying tokens, and their decimal precision are fixed through constructor configuration. Deployment checks verify that the token pair matches the adapter and that the provider references the same adapter.

When peek is called for the configured provider share, the oracle retrieves the outstanding share supply and requests the position's composition at the adapter's fair price. This composition includes deployed principal, tracked idle inventory, and accrued trading fees. Each underlying token amount is valued using its resilient oracle price and decimal precision. The aggregate value is then allocated across the outstanding shares, normalized to one whole share, and reduced by the configured haircut.

The pricing flow distinguishes between zero supply, unavailable underlying prices, and a calculated share price that rounds to zero. Zero share supply returns zero immediately. With outstanding shares, a zero price for either underlying token causes valuation to revert. If the final calculated share price is zero, the contract returns one oracle-price unit, corresponding to USD 0.00000001 per whole share.

For tokens other than the configured provider share, peek forwards the request to the resilient oracle. Token-configuration requests follow the same distinction: the provider share identifies this contract as its sole enabled price source, while other tokens use the resilient oracle's configuration. This allows a Moolah market to obtain both provider-share collateral pricing and other required token prices through the same interface.

Initialization establishes the administrator, manager, resilient oracle, and initial haircut. The MANAGER role may adjust the haircut within a maximum of 10%, while implementation upgrades require the default administrator role. The oracle performs valuation only; liquidity management and share accounting remain with the adapter and provider.

Privileged Functions

- setHaircutBPS

Issue_24	Constructing the share oracle before provider initialization snapshots incorrect share decimals
Severity	Low
Description	The `V3ProviderOracle` constructor reads the provider share's `decimals()` and stores the result in the immutable `SHARE_DECIMALS`. Subsequent calls to `peek()` use this captured value to normalize the portfolio valuation into the price of one whole provider share. However, the provider proxy's share decimals are established during ERC-4626 initialization. Before `__ERC4626_init()` executes, the share implementation returns zero decimals. Constructing the oracle implementation against an existing but uninitialized provider proxy can therefore capture zero instead of the provider's eventual decimal precision. The constructor's adapter-wiring check does not exclude this ordering. The provider's `ADAPTER()` getter exposes an implementation immutable and can return the expected address before the provider's storage has been initialized. **Passing the adapter check therefore does not establish that the share-decimal value is ready to be captured.** Initializing the provider afterward updates its live share decimals but does not update the oracle implementation's immutable snapshot. For a provider intended to use 18 share decimals, the oracle continues applying a zero-decimal normalization. The resulting collateral quotation is materially understated and can reach the minimum positive oracle price. Meaningful borrowing against the affected shares can consequently remain unavailable because Moolah interprets the quotation using the provider's live decimals. This genesis scenario does not establish overborrowing or liquidation of pre-existing material debt: the incorrect quotation is present from the market's initial use. Debt-free collateral withdrawals retain their ordinary exit path. The condition persists for the affected oracle implementation and

	requires replacement with an implementation constructed against the correctly initialized provider.
Recommendations	Consider initializing the proper before oracle construction.
Comments / Resolution	Resolved.

<table>
<tr><td>Issue_25</td><td>Increasing the haircut can immediately make existing positions liquidatable</td></tr>
<tr><td>Severity</td><td>Low</td></tr>
<tr><td>Description</td><td>
<code>V3ProviderOracle.peek()</code> values provider shares and then applies the configured <code>haircutBps</code>. Moolah uses the resulting discounted price when assessing collateral sufficiency and liquidation eligibility.
The <code>MANAGER</code> can change this haircut through <code>setHaircutBps()</code>, up to a maximum of 10%. The updated value applies immediately to subsequent oracle reads. The oracle contract does not provide a transition period or preserve the previous haircut for existing borrowers.
Consequently, increasing the haircut can make an otherwise unchanged position unhealthy. Neither a decline in the underlying assets nor an increase in the borrower's debt is necessary.
For example:
<table>
<tr><th>Metric</th><th>Before the change</th><th>After the change</th></tr>
<tr><td>Collateral value before haircut</td><td>$10,000</td><td>$10,000</td></tr>
<tr><td>Haircut</td><td>0%</td><td>5%</td></tr>
<tr><td>Oracle-valued collateral</td><td>$10,000</td><td>$9,500</td></tr>
<tr><td>Liquidation loan-to-value</td><td>80%</td><td>80%</td></tr>
<tr><td>Maximum healthy debt</td><td>$8,000</td><td>$7,600</td></tr>
<tr><td>Actual debt</td><td>$7,800</td><td>$7,800</td></tr>
<tr><td>Position status</td><td>Healthy</td><td>Liquidatable</td></tr>
</table>
</td></tr>
</table>

	The borrower initially has sufficient collateral. Once the haircut increases, the unchanged debt exceeds the permitted threshold by **$200**. The parameter update does not itself execute a liquidation. However, a subsequent permitted liquidation can seize collateral and apply the liquidation incentive before the borrower adjusts the position. This behavior exposes existing borrowers to liquidation following an authorized configuration change. It is an expected consequence of changing a collateral-risk parameter, rather than an oracle arithmetic error or an access-control bypass. Any delay enforced by external governance would need to be assessed separately. The impact can extend to lenders where the market's LLTV and existing debt are sufficiently high. Moolah calculates liquidation repayment using the discounted collateral quotation. If a liquidation seizes all remaining collateral, any outstanding borrower debt is written off against the market's `totalSupplyAssets`. However, the seized provider shares remain redeemable for their proportional underlying assets without applying the oracle haircut. For example, assume a market with **90% LLTV**, **$100,000 of redeemable collateral backing**, **$89,000 of debt**, and an initial haircut of zero. Increasing the haircut to **10%** reduces the collateral quotation to **$90,000** and makes the position liquidatable. Under Moolah's liquidation incentive, seizing all collateral requires approximately **$87,300 of repayment**. The remaining **$1,700 is written off against lenders**, while the liquidator receives shares still redeemable for approximately **$100,000**. Consequently, a haircut increase can cause an actual lender loss despite sufficient underlying backing to cover the original debt. This consequence depends on the market configuration and borrower leverage; it does not follow from every haircut increase.
Recommendations	Consider announcing any haircut increase with sufficient time before execution.

Comments / Resolution	Acknowledged.

Issue_26	Minimum positive oracle pricing can overvalue shares with backing below the minimum quote
Severity	**Low**
Description	For a nonzero share supply, `V3ProviderOracle.peek()` calculates the share price, applies the haircut, and replaces a resulting zero with `1`. That value represents **$0.00000001 per whole provider share**, not per smallest share unit. Zero total supply returns zero, while a zero underlying token-price feed still causes a revert. When actual recoverable backing per whole share falls below that minimum quotation, the oracle can report more collateral value than the shares can redeem. **The absolute overvaluation grows with the number of whole shares.** It is therefore incorrect to assume that returning one oracle unit limits the total valuation error to one oracle unit. While we could not establish a definitive scenario where share prices go below that value, it would expose a critical issue if that ever happens.
Recommendations	We recommend moving back to the previous version and reverting if the value is zero. In the worst case where this ever happens, the contract can be upgraded. However, we are the opinion that this is more safe from a hypothetical perspective.
Comments / Resolution	Resolved.

V3ProviderLib

The V3ProviderLib library centralizes the deposit-quotation and USD-valuation calculations used by V3Provider. Its externally linked functions reduce the provider's deployed bytecode size. The provider supplies the relevant adapter, oracle, token addresses, and token decimals through a context structure. The library maintains no independent accounting state and performs no token transfers, share issuance, or liquidity operations.

The amountsValueUsd function retrieves both underlying token prices from the configured resilient oracle and converts the supplied amounts into an aggregate eight-decimal USD value. Each token's decimals are applied separately, with fractional USD units rounded down. A zero price for either token causes the calculation to revert, including when the corresponding token amount is zero. The provider uses this calculation when valuing an initial liquidity contribution and the managed position.

The quoteDeposit function calculates the accepted token amounts and corresponding shares for deposits made while shares already exist. It obtains the adapter's position composition at the current pool spot price, including principal, tracked idle inventory, and accrued fees. After verifying that the composition has a nonzero oracle valuation, it determines the proportional contribution supported by the depositor's desired amounts. For a two-token composition, the more restrictive token determines this proportion. For a one-token composition, the existing token leg determines it.

The contribution proportion and resulting share amount are rounded down, while the accepted token amounts are rounded up. The oracle valuation serves as a nonzero-value check; the share calculation itself follows the live token composition. This quotation logic is shared by subsequent-deposit execution and the provider's share-preview function. The provider separately enforces minimum amounts and share output, rejects zero-share deposits, transfers accepted assets into idle inventory, and supplies the issued shares to Moolah.

Privileged Functions

- none

Issue_27	Residual supply with zero rounded USD value blocks new deposits
Severity	Medium
Description	For subsequent deposits, `V3ProviderLib.quoteDeposit()` first values the provider's existing token composition and reverts with `ZeroShares` if that value is zero. This check occurs before the proposed contribution is incorporated into the position. The valuation can round to zero even when the token balances and oracle prices are nonzero. `_amountsValueUsd()` converts each token leg into eight-decimal USD units separately, rounding down each result. For example, an 18-decimal token priced at $600 requires at least 16,666,667 base units to contribute one unit of eight-decimal USD value. A smaller balance contributes zero. If both remaining token legs fall below their respective thresholds, the combined reported value is also zero. Near-total withdrawals can leave this state while a positive residual share supply remains outstanding. A subsequent user cannot reopen the provider through the initial-deposit branch because that branch requires `totalSupply()` to equal zero. Instead, the existing zero-value composition causes the subsequent-deposit path to revert, regardless of whether the proposed contribution would be economically significant. `previewDepositShares()` propagates the same failure. The condition can persist when the remaining shares belong to an inactive holder. The impact is temporary closure of the ordinary deposit path in a nearly empty provider. Deposits become available again if the relevant state changes sufficiently, for example, through fee accrual or a successful liquidity addition to the tracked NFT. A plain ERC-20 transfer to the adapter does not update its tracked idle inventory and therefore does not necessarily resolve the condition. No assets are lost through the failed deposits. The issue is a residual-state availability limitation caused by requiring the existing portfolio to

	have a positive rounded USD value before accepting new backing.
Recommendations	Consider ensuring that governance executes and retains the first deposit.
Comments / Resolution	Resolved, governance will execute the first deposit and keep it.

V3PositionLib

The V3PositionLib library centralizes the position-manager interactions used by V3DexAdapter. It provides externally linked functions for creating positions, increasing and decreasing liquidity, collecting owed tokens, and burning empty position NFTs. These functions execute in the calling adapter's context, so token approvals, NFT ownership, and collected assets remain associated with the adapter.

The mint and increaseLiquidity functions increase the position manager's allowances for both underlying tokens and forward the desired amounts and minimum-consumption requirements. New positions are minted directly to the adapter. Both functions return the resulting liquidity and actual token amounts consumed, allowing the adapter to account for unused inputs.

The decreaseLiquidity function forwards the requested liquidity reduction and token minimums to the position manager. The resulting assets become collectible through the position manager and are retrieved separately through collectAll. Collection requests the maximum available amount of both tokens and directs the proceeds to the adapter. The burn function removes an empty position NFT through the position manager.

Position-manager deadlines are set to the current block timestamp. Access restrictions, operation-level deadlines, share calculations, idle-inventory accounting, and updates to the active position identifier are handled by the calling contracts. The library itself maintains no independent position state.

Privileged Functions

- none

No issues found.

SwapInventoryLib

The SwapInventoryLib library implements the inventory-conversion operation used during adapter rebalancing. It executes externally supplied swap instructions against a selected venue

and returns updated token totals based on measured balance changes. As an externally linked library, it operates in the adapter's context and uses the adapter's assets and allowances.

The swap function selects the input and output tokens according to the requested conversion direction. A zero input amount or empty input inventory returns the existing totals unchanged. Otherwise, the requested input must not exceed the available inventory supplied by the adapter.

For ERC-20 inputs, the library grants the venue an allowance equal to the requested amount, executes the supplied calldata, and clears the remaining allowance. For native-currency inputs, the input token must be the configured wrapped-native asset. The library unwraps the requested amount and forwards it with the external call. When either side of the pair is the wrapped-native asset, additional native currency received during execution is wrapped before settlement is measured.

After execution, the library determines the actual input spent and output received from token-balance differences. It rejects spending above the requested input and output below the specified minimum. The inventory totals are adjusted by these measured amounts and returned to the adapter.

Venue authorization and reference-price loss limits are enforced by V3DexAdapter around the library call. SwapInventoryLib is responsible for execution and balance-based settlement, while the adapter determines which conversions are permitted and how the resulting inventory is deployed.

Privileged Functions

- none

Issue_28	Removing the managed position can invalidate a swap quote that routes through the same pool
Severity	**Medium**
Description	The full rebalance removes the existing NFT's liquidity and collects the underlying assets **before** executing the inventory-conversion swap. A swap quote obtained against the pool's pre-rebalance state can

	therefore assume liquidity that will no longer exist when the conversion executes. Although `setSwapPairWhitelist()` prevents the managed pool itself from being whitelisted as the direct execution target, it permits approved routers and aggregators. The adapter does not inspect their internal routes to establish whether those routes pass through the managed pool. **Rejecting the pool as a direct call target does not prevent a whitelisted router from using it.** For standard V3 pool behavior, removing active liquidity changes execution depth without itself moving the current square-root price. Consequently, an unchanged starting price does not establish that the original swap quote remains valid. The relevant precondition is that the keeper's quote reflects the state **before position removal**, rather than the state produced by the preceding operations of the same rebalance. This will have second order impacts such as slippage decrease and price changes from the swap execution.
Recommendations	Consider ensuring that no aggregator or router routes via the own V3 position.
Comments / Resolution	Resolved. This will be ensured

V3Liquidator

Disclaimer: The V3Liquidator has only been audited in context with the scope in this report. Any usage on contracts outside of this scope is not covered.

V3Liquidator

The V3Liquidator contract is an upgradeable liquidation executor for Moolah markets using V3Provider shares as collateral. It coordinates collateral seizure, optional redemption into the underlying pool tokens, conversion into the loan token, and repayment to Moolah. Operational functions are restricted to the BOT role, while configuration and asset-management permissions are assigned separately.

The basic liquidate function requests liquidation in an approved market and retains the seized provider shares within the liquidator. Repayment uses the contract's existing loan-token balance. When a shared funding vault is configured, the liquidation callback requests only the additional amount needed to cover repayment. Moolah determines liquidation eligibility, the collateral seized, and the required repayment.

The flashLiquidate function supports redeeming seized shares within Moolah's liquidation callback. Moolah transfers the collateral before invoking the callback and collects repayment afterward. The liquidator can therefore redeem the shares, exchange the resulting assets, and approve repayment within the same transaction. This path validates the provider against its whitelist and the market's collateral token, together with the selected swap venues and any separate token-approval targets.

Redemption forwards the specified minimum token outputs to the provider. The received assets may then be exchanged using BOT-supplied swap instructions. Wrapped-native collateral is received as native currency and is either exchanged directly or wrapped again when it is also the repayment asset. The redeeming callback requires the incremental loan-token proceeds to cover the repayment amount, excluding pre-existing holdings from this calculation. Break-even execution satisfies this requirement. When redemption is disabled, the callback instead uses existing funds and the optional funding-vault mechanism.

The contract also provides standalone functions for redeeming previously acquired provider shares and selling held ERC-20 tokens or native currency. Standalone sales require approved tokens and venues, measure actual token movements, and enforce maximum input and minimum output requirements. When a funding vault is configured, standalone redemptions must deliver their proceeds to the liquidator itself. Following the applicable liquidation and redemption operations, the full remaining balances of designated assets are transferred to the funding vault, subject to a configurable reflow blacklist.

MANAGER controls the market, provider, token, and venue whitelists, as well as the funding source and reflow blacklist. Both MANAGER and the configured funding source may withdraw assets to themselves. Operational liquidation, redemption, and sale functions are protected against reentrancy, while the liquidation callback accepts calls only from Moolah. Implementation upgrades require the default administrator role.

Privileged Functions

- withdrawERC20
- withdrawETH
- setFundSource

- setReflowBlacklist
- setTokenWhitelist
- setMarketWhitelist
- batchSetMarketWhitelist
- setPairWhitelist
- setV3ProviderWhitelist
- batchSetV3Providers
- liquidate
- flashLiquidate
- redeemV3Shares
- sellToken
- sellToken
- sellBNB

Issue_29	Governance Privilege: Control over funds
Severity	Governance
Description	It is apparent that the BOT role within this contract is more trusted than within the actual provider, as the BOT has obvious ways to withdraw inventory from the liquidator via multiple different venues such as providing malicious swap data or redeeming V3 shares to arbitrary addresses.
Recommendations	Consider acknowledging this trust assumption.
Comments / Resolution	Acknowledged.

Issue_30	Wrapped-native liquidation can bypass native-balance retention
Severity	Low
Description	`V3Liquidator.setReflowBlacklist()` allows the manager to exclude individual assets from automatic transfers to `fundSource`. When the native-token identifier is blacklisted, `_reflow(BNB_ADDRESS)` leaves the liquidator's native balance

	untouched. However, a subsequent liquidation can convert that retained balance into an asset governed by a different blacklist entry. When `onMoolahLiquidate()` processes a redemption whose loan token equals the provider's wrapped-native token, it wraps the liquidator's entire native balance. This includes native currency retained from previous operations. After repayment, `flashLiquidate()` calls `_reflow()` for the loan token. This checks only the wrapped-native token's blacklist entry. Consequently, if native BNB is blacklisted but WBNB is not, the previously retained native balance is transferred to `fundSource` as WBNB. For example, assume the liquidator retains 10 BNB under the native blacklist. A subsequent WBNB-denominated liquidation generates another 5 BNB against a repayment obligation of 4 WBNB. The callback wraps all 15 BNB, repays 4 WBNB, and transfers the remaining 11 WBNB to `fundSource`. The original 10 BNB is therefore no longer retained locally. This requires a configured `fundSource`, an eligible liquidation with share redemption, and different blacklist settings for the native and wrapped representations. The existing native balance is included in the callback's initial profitability snapshot. It therefore cannot disguise insufficient liquidation proceeds. The impact is limited to bypassing the intended local retention policy; the funds remain in protocol custody.
Recommendations	Consider keeping that in mind.
Comments / Resolution	Acknowledged.

V3Stable

StableV3Provider

The StableV3Provider contract is the concrete V3Provider implementation intended for stable-token liquidity positions. It inherits the base provider's share accounting, deposit and redemption interfaces, Moolah collateral integration, whitelist controls, and compounding functionality. Its initializer establishes the administrative roles, resilient oracle, accounting asset, and share-token metadata.

The contract exposes a BOT-restricted rebalance function that forwards execution parameters to the inherited guarded-rebalance flow. These parameters include minimum withdrawal

amounts, minimum replacement liquidity, an optional target pool price, an expected reference rate, a deadline, and optional inventory-conversion instructions. Execution is protected against reentrancy and remains subject to the provider's configurable per-rebalance and cumulative daily loss limits.

Position management and reference-price construction are delegated to the associated adapter. Share issuance and asset valuation follow the inherited V3Provider accounting mechanisms.

Privileged Functions

- rebalance

No issues found.

StableV3DexAdapter

The StableV3DexAdapter contract specializes V3DexAdapter for stable-token pairs by deriving its reference exchange rate from resilient oracle prices. It stores the oracle address immutably and reads the USD prices of both underlying tokens to determine their relative value. A zero price for either token causes the rate lookup to revert.

The resulting reference represents the amount of token1 corresponding to one whole token0. The inherited adapter converts this reference into the pool's price representation while accounting for differences in token decimals. It also constructs the liquidity range around the reference using independently configurable lower and upper margins, with boundaries aligned to the pool's tick spacing.

Initialization accepts the range margins and permitted spot-price deviation. It records the current oracle-derived rate as the last center rate and initializes the minimum rate movement

required for rebalancing to one basis point. This rate-movement threshold is separate from the configurable spot-price deviation tolerance. The reference rate also supplies the inherited compounding price check, rebalance rate checks, and inventory-conversion loss limit.

When the provider is assigned, the adapter verifies reciprocal adapter configuration and requires the provider to use the same resilient oracle. Liquidity custody, fee collection, idle-inventory accounting, withdrawals, compounding, and position replacement are inherited from V3DexAdapter. Operational calls remain restricted to the configured provider, MANAGER controls strategy parameters, and upgrades require the default administrator role.

Privileged Functions

- none

No issues found.

V3SlisBNB

SlisBNBV3Provider

SlisBNBV3Provider

The SlisBNBV3Provider contract specializes V3Provider for slisBNB/WBNB liquidity positions and integrates the associated Moolah collateral with the slisBNBx reward-accounting system. It inherits the base provider's deposits, share issuance, withdrawals, redemptions, and access controls. Its additional responsibilities are tracking collateral shares across Moolah markets, exposing their BNB-denominated value, and notifying the configured slisBNBx minter when synchronization occurs.

The contract maintains a per-user, per-market record of synchronized collateral shares in userMarketDeposit and an aggregate share amount in userTotalDeposit. These records represent provider shares deposited as Moolah collateral. During synchronization, the contract

reads the current collateral amount from Moolah, applies the difference to the user's aggregate, and updates the market-specific record.
The inherited collateral-change hook invokes this synchronization after the relevant provider operations and Moolah liquidation notifications. Permissionless single-user and batch synchronization functions also allow the records to be refreshed. Each supplied market must use this provider as its collateral token, and batch synchronization requires matching market and account arrays.

The getUserBalanceInBnb function values the user's synchronized collateral shares against the adapter's fair-price composition, including idle assets and accrued fees. It values the proportional token amounts through the resilient oracle and converts their combined USD value into BNB using the oracle's native-BNB price. The result uses 18-decimal BNB units. Zero recorded collateral or zero total share supply returns zero, while a zero required oracle price causes valuation to revert.

After updating collateral records, the provider calls the configured slisBNBx minter. That separate contract manages reward-token adjustments according to its module discount, fee allocation, and delegation settings. MANAGER controls the minter address; a zero address disables these external minter calls.

The contract additionally exposes a BOT-restricted, nonreentrant rebalance entry point. Execution is forwarded through the inherited guarded-rebalance mechanism and remains subject to the provider's per-operation and cumulative daily loss limits.

Privileged Functions

- setSlisBNBxMinter

Issue_31	Rebalancing can deploy liquidity at the redemption-rate anchor during a market depeg
Severity	**High**
Description	`SlisBNBV3DexAdapter` derives its reference price from the StakeManager's slisBNB-to-BNB conversion rate. This rate represents the protocol's redemption accounting and can remain above the price obtainable through immediate secondary-market trading. During rebalancing, the adapter constructs the replacement range around this rate and deploys the available inventory at the pool's execution price. The final price assertion compares spot against the BOT-supplied target; it does not independently establish that this target reflects current secondary-market value.

Consequently, a pool price temporarily restored to the redemption-rate anchor can be accepted while slisBNB continues trading at a discount elsewhere.

For example:

Reference	Value of 1 slisBNB
StakeManager anchor	1.2 BNB
Secondary-market value	1.1 BNB

At an active pool price of **1.2 BNB**, the position initially offers approximately **0.1 BNB more per slisBNB** than the secondary market, before fees and price impact. As trading moves the pool toward **1.1 BNB**, the position pays out BNB and accumulates slisBNB.

The provider consequently purchases slisBNB above its immediate market value. The total effect depends on the position's range, deployed inventory, price path, and trading fees; the initial price difference cannot be applied directly to the entire portfolio.

The provider's NAV limits remain enforceable, but measure the before-and-after change using its configured valuation framework. Where that framework remains anchored to the redemption rate, it does not independently constrain the subsequent market-value effect of arbitrage after the rebalance has completed.

The resulting exposure is temporary or lasting underperformance relative to retaining the original inventory. Market recovery can reverse part of an unchanged liquidity position's composition effect; withdrawals or subsequent strategy changes can instead crystallize the unfavorable outcome.

Recommendations

Consider not rebalancing and compounding if the market rate of SlisBNB differs from the StakeManager's anchor rate.

Comments / Resolution	Acknowledged. The team is aware of this behavior and will ensure that the BOT only rebalances if the anchor rate matches with the market rate.

Issue_32	Rate-anchored collateral valuation may overstate recoverable value during a slisBNB depeg
Severity	**Medium**
Description	`SlisBNBV3DexAdapter` derives its reference price from the conversion rate returned by `StakeManager.convertSnBnbToBnb()`. This rate represents slisBNB's staking conversion value rather than necessarily the price at which the provider's holdings can immediately be exchanged for BNB. `V3ProviderOracle` uses this reference price to calculate the liquidity position's underlying composition. It then values that composition, together with fees and idle inventory, through the configured resilient oracle and applies the configured collateral haircut. Where the underlying slisBNB price feed is also derived from the staking conversion rate, a secondary-market discount need not cause a corresponding reduction in the reported share price. The collateral

	quotation can therefore continue to reflect the staking reference while immediate redemption and sale of the actual underlying assets would recover less. Moolah uses the resulting share price to determine borrowing capacity and liquidation eligibility. Consequently, borrowers may obtain or retain more debt than a valuation based on executable recovery proceeds would support. Positions may remain healthy under the oracle despite weakened economic coverage. The discrepancy becomes material when timely redemption at the staking conversion rate is unavailable, constrained, or costly, and secondary-market liquidity cannot provide equivalent recovery. A conversion quote alone does not establish that the entire collateral position can be converted at that value within the required liquidation period. If the discrepancy exceeds the available risk buffers, liquidators may be unable to recover sufficient value to cover repayment and execution costs. This can delay liquidation or leave lenders exposed to bad debt. The relevant buffers include the oracle haircut, market LLTV, and the margin required to support the liquidation incentive. A slisBNB discount does not translate directly into the same percentage loss for the entire provider. Actual exposure depends on the position's composition, idle balances, redemption routes, market depth, and transaction costs. A modest discount may only consume existing risk headroom. This concern applies to the combined adapter, underlying price feeds, and lending-market configuration. The adapter does not independently determine the slisBNB USD price, and a depeg alone does not establish undercollateralized borrowing without comparing the complete collateral quotation against realistically recoverable proceeds.
Recommendations	Consider implementing emergency mechanisms if that ever occurs, there could be additional second-order effects such as reverting liquidations due to the BOT calculating wrong slippage from the anchor

	rate.
Comments / Resolution	Acknowledged.

Issue_33	Minter replacement can duplicate allocations for unchanged collateral
Severity	**Low**
Description	`SlisBNBV3Provider.setSlisBNBxMinter()` allows the MANAGER to replace the minter used for subsequent collateral synchronization. The function updates the configured address without reconciling the allocations previously issued by the old minter. Each `SlisBNBxMinter` maintains independent `userModuleBalance` and `userTotalBalance` records. A newly initialized replacement therefore has no record of allocations already issued against the provider's collateral. During synchronization, it calculates the full current entitlement and treats that amount as newly mintable. Where the replacement is authorized to mint the same slisBNBx token, has the provider registered as an enabled module, and has sufficient reserve capacity, unchanged collateral can consequently receive a second allocation. For example, assume collateral worth **100 BNB**, no discount, and a **3%**

	reserve fee: | Allocation	Old minter	Replacement minter	Combined
---	---	---	---
User allocation	97 slisBNBx	97 slisBNBx	194 slisBNBx
Reserve allocation	3 slisBNBx	3 slisBNBx	6 slisBNBx
Total	100 slisBNBx	100 slisBNBx	200 slisBNBx	 The collateral remains worth **100 BNB**, while the two minters collectively account for **200 slisBNBx**. Subsequent collateral reductions are forwarded only to the currently configured minter. A full withdrawal can therefore remove the replacement's allocation while leaving the old allocation outstanding until separately reconciled. Overlapping minter registration also matters independently of the pointer update: `syncUserModuleLp()` does not verify that the provider currently selects that minter. Merely changing the provider's configured address does not invalidate the previous minter's accounting. This can inflate the outstanding representation of eligible collateral and potentially distort rewards where the external reward system recognizes both allocations. It does not create additional V3 shares or additional Moolah borrowing capacity. The issue concerns replacement with an independently initialized minter. An upgrade preserving the existing minter's storage does not inherently duplicate allocations. Actual reward impact and the external token's minting permissions require deployment-level confirmation.
Recommendations	Consider not changing the minter.

Comments / Resolution	Acknowledged.

Issue_34	MCP caps may result in afterCollateral revert
Severity	**Low**
Description	The SlisBNB mechanism incorporates mcp wallets which are exposed to caps. If these caps are exceeded, the transaction fully reverts, essentially freezing the provider.
Recommendations	Consider keeping caps always sufficiently high.
Comments / Resolution	Acknowledged.

SlisBNBV3DexAdapter

The SlisBNBV3DexAdapter contract specializes V3DexAdapter for the configured slisBNB/WBNB pair on BNB Chain. Its constructor requires slisBNB as token0 and WBNB as token1, and configures WBNB as the wrapped-native asset. The slisBNB, WBNB, and StakeManager addresses are defined as contract constants.

The adapter obtains its reference exchange rate by requesting the BNB value of one whole slisBNB from StakeManager. This rate supplies the inherited fair-price calculation, liquidity-range construction, and applicable inventory-conversion loss checks. The lower and upper range margins are supplied during initialization and remain adjustable through the inherited management functions.

Initialization records the current exchange rate as the last center rate and sets the minimum relative rate movement required for rebalancing to one basis point. The separate spot-price deviation tolerance is initialized by the base adapter to 50 basis points and can be adjusted by MANAGER.

NFT custody, liquidity additions and removals, fee collection, idle accounting, compounding, and rebalancing are implemented by V3DexAdapter. Inventory conversions use the inherited whitelisted-venue mechanism, through which supported staking or redemption venues may be

selected. Native currency is accepted from the wrapped-native contract or an approved swap venue. Operational calls are restricted to the configured provider, and implementation upgrades require the default administrator role.

Privileged Functions

- none

No issues found.

SlisBNBV3ProviderOracle

SlisBNBV3ProviderOracle

The SlisBNBV3ProviderOracle contract is the slisBNB/WBNB-specific implementation of V3ProviderOracle. Its constructor restricts the underlying pair to the configured slisBNB address as token0 and WBNB address as token1. The inherited constructor additionally verifies that the adapter uses the same pair and that the provider share references that adapter.

Share pricing follows the inherited oracle mechanism. The contract retrieves the position's fair-price composition from the adapter, values the underlying amounts through the resilient oracle, allocates the total value across outstanding shares, and applies the configured haircut. The result is an eight-decimal USD price per whole provider share. With the corresponding slisBNB adapter, the composition reference originates from StakeManager's slisBNB-to-BNB conversion rate.

The contract inherits the zero-supply return, rejection of zero underlying-token prices, and minimum positive price applied when the final share valuation rounds to zero. Pricing and token-configuration requests for other assets are forwarded to the resilient oracle.

Initialization and administrative controls are inherited from V3ProviderOracle. MANAGER may adjust the haircut up to the 10% limit, while the default administrator authorizes implementation upgrades.

Privileged Functions

- none

No issues found.